

CodeMentor AI

Socratic Coding Tutor — Complete Technical Specification

Course: UE23CS342BA9 — Generative AI and its Applications

4 Credits · 56 Hours

1. Project Overview

CodeMentor AI is a conversational AI tutoring system designed to help students solve LeetCode problems without giving them the answer. It follows the Socratic method — asking guiding questions, identifying where the student is stuck, and nudging them toward the solution through their own reasoning.

The system is explicitly scoped to one problem per session. The student may reference any LeetCode problem by number, title, function signature, or natural language description. All questions in the session are assumed to be about that one problem.

1.1 Core Principles

- The bot never reveals the answer, solution code, or direct fix under any circumstance.
- Every response is a question or a gentle directional nudge.
- The bot diagnoses what the student actually needs, not just what they say they need.
- If a student pressures for the answer, the bot redirects by asking what part is confusing.
- One LeetCode problem per session — the problem is identified once and stored.

1.2 Scope Constraints

The following are deliberate scope limitations for this version:

- Students are assumed to be asking about LeetCode problems only — no general conceptual or syntax questions unrelated to a problem.
- One problem per conversation session. The system does not switch problems mid-session.
- No file uploads — students paste code directly in the chat.
- No execution environment — the system cannot run code.

2. Technology Stack

Component	Technology	Purpose
LLM Inference	Groq API — Llama 3.1 8B Instant	All agent and response generation calls
Vector Database	ChromaDB (local, persistent)	Problem and concept document storage
Embedding Model	sentence-transformers all-MiniLM-L6-v2	Converts text to vectors for similarity search
Backend Framework	FastAPI (Python)	REST API, session management, pipeline orchestration
Streaming	FastAPI StreamingResponse + SSE	Token-by-token response streaming to frontend
Frontend	Chainlit	Student chat interface
Deployment (backend)	Railway.app or Render (free tier)	Hosts FastAPI server and ChromaDB files
Deployment (frontend)	Vercel (free)	Hosts the student-facing chat interface

Deployment may be done later, first the system must be made to run locally

3. Dataset

3.1 LeetCode Problem Dataset

Original Dataset Link:

<https://github.com/neenza/leetcode-problems>

The dataset contains LeetCode problems used as the retrieval layer in the RAG system. Each document stored in ChromaDB contains the following selected fields from the dataset:

- **title:** The name of the problem (e.g., *"Container With Most Water"*)
- **frontend_id:** The LeetCode frontend ID (string)
- **difficulty:** The difficulty level (*Easy, Medium, or Hard*)
- **problem_slug:** The URL-friendly name (e.g., *container-with-most-water*)
- **topics:** Array of topic tags (e.g., *Array, Two Pointers*)
- **description:** The full problem statement (usually in Markdown format)
- **examples:** Array of example objects, each containing:
 - **example_num:** Example number
 - **example_text:** Input/output and explanation
 - *(Note: Image URLs are skipped)*
- **constraints:** Array of constraints or limits for the problem
- **hints:** Array of hints for solving the problem
- **code_snippets:** Starter code for various languages (*only Python is retained*)
- **solutions:** HTML string containing editorial content for some problems

3.2 What Gets Embedded for ChromaDB

Each problem is stored as a single document in ChromaDB. The text that is embedded (converted into a vector for semantic search) is a concatenation of the most relevant searchable fields:

```
embed_text = f"{problem['id']} {problem['title']}  
{problem['function_signature']} {problem['description'][:500]}"
```

This ensures that vector search can effectively match across all four input types:

- Problem number
- Title
- Function name
- Natural language description

3.3 Solution Approach Field Usage

The **solutions** field is never shown to the student. It is used exclusively by the Code Agent (Agent 3) as a reference to:

- Understand the gap between the student's approach and the correct one
- Guide the student without directly revealing the answer

It is important to note that:

- There may be multiple valid approaches to solving a problem
 - The solutions field represents only some of the possible correct approaches
-

4. System Architecture Overview

The system is a deterministic pipeline with three LLM agents that fire conditionally. Every student message passes through the same top-level structure but takes different internal paths based on decisions made at each stage.

Pipeline summary (one student message, worst case):

1. Problem identification → Rule-based check + optional Query Rewriter Agent + ChromaDB RAG
2. Intent classification → Intent Classifier Agent (always fires)
3. Code analysis → Code Agent (fires only for code_help intent)
4. Response generation → Dynamic few-shot + Prompt builder + Groq LLM call (streamed)

Worst case: 4 Groq API calls per message (query rewriter + intent classifier + code agent + response generator). Expected latency under 2 seconds before first streamed token appears.

5. Problem Identification Pipeline

This pipeline runs at the start of every message turn. Its job is to ensure a problem is in the session before intent classification proceeds. Once a problem is in session, this entire pipeline is skipped — it only runs once per conversation.

5.1 Full Decision Tree

```
Student message arrives
|
v
[ STEP 1 ] Is problem already in session?
  YES → skip to intent classification (section 6)
  NO  → continue below
      |
      v
[ STEP 2 ] Is a problem statement pasted directly?
            (rule-based check – no LLM call)
  YES → store it directly from the user prompt

  NO  → continue below
      |
      v
[ STEP 3 ] Does message reference a problem?
            (problem number / title / function name / description)
  YES → Query Rewriter Agent → ChromaDB search → confidence check
        high confidence → store in session → intent classification
        low confidence  → ask student: "Did you mean [X]? Paste if not"
        not found       → ask student: "Please paste the problem
statement"
  NO  → ask student: "Which LeetCode problem are you working on?"
```

5.2 Step 2 — Problem Statement Paste Detection (Rule-Based)

This is a fast rule-based check. No LLM call. If the message passes this check, the raw text is stored as the problem in session and RAG is skipped entirely. This handles problems not in the dataset.

```
def is_problem_statement(message: str) -> bool:
    if len(message) < 200:
        return False
    keywords = ['example', 'input:', 'output:', 'constraints:', 'return ',
                'given', 'note:', 'follow-up', 'explanation']
    message_lower = message.lower()
    matches = sum(1 for kw in keywords if kw in message_lower)
    return matches >= 3
```

5.3 Step 3 — Problem Reference Detection (Rule-Based)

Determines if the message is referencing a known LeetCode problem without pasting the full statement.

```
def references_problem(message: str) -> bool:
    import re
    # Matches: 'problem 1', '#42', 'lc 200', 'leetcode 1'
    if re.search(r'(problem\s*#\d+|lc\s*\d+|leetcode\s*\d+|#\d+)', message,
re.I):
        return True
    # Matches camelCase function names: twoSum, maxProfit
    if re.search(r'\b[a-z]+[A-Z][a-zA-Z]+\b', message):
        return True
    # Message is substantial enough to be a description
    if len(message) > 40:
        return True
    return False
```

COMMENT: see if step 2 and 3 could also be replaced by and agent/llm call because rule based check might fail. How would the project structure change in that case? First explain before making changes.

5.4 Query Rewriter Agent (Agent 1)

Fires when: Step 3 determines a problem is referenced but it is not in session.

Input: Raw student message.

Output: A clean 2-5 word search query optimized for ChromaDB vector search.

Model: llama-3.1-8b-instant via Groq. Max tokens: 30.

```
QUERY_REWRITER_PROMPT = """
Extract the LeetCode problem being referenced from this student message.
Return ONLY a short search query (2-5 words) – the problem title or
a brief description of what the problem asks. Nothing else.

Examples:
Message: 'I am working on problem 1' → 'two sum array target'
Message: 'def twoSum(self, nums, target):' → 'two sum array target'
Message: 'the one where you find islands in a grid' → 'number of islands
grid'
Message: 'Two Sum' → 'two sum array target'

Student message: {message}
Search query: ""
```

COMMENT: Add more examples to this prompt of other lc questions.

5.5 ChromaDB RAG Search

The clean query from Agent 1 is embedded using sentence-transformers and searched against the LeetCode problems collection in ChromaDB.

```
def search_problem(query: str) -> tuple[dict | None, float]:
    embedding = embed_model.encode(query).tolist()
    results = problems_collection.query(
        query_embeddings=[embedding],
        n_results=1,
        include=['documents', 'metadatas', 'distances']
    )
    if not results['ids'][0]:
```

```

        return None, 0.0
    # ChromaDB returns L2 distance – convert to similarity score
    distance = results['distances'][0][0]
    score = 1 / (1 + distance)
    problem_data = results['metadatas'][0][0]
    return problem_data, score

CONFIDENCE_THRESHOLD = 0.75

```

5.6 Fallback Responses

Situation	Bot Response
No problem referenced	"Which LeetCode problem are you working on? You can share the problem number, title, or paste the problem statement directly."
Low confidence match	"I think you might be asking about [Problem Title]. Here's what I found: [brief description]. Is this the right problem? If not, please paste the problem statement directly."
Not found in dataset	"I couldn't find that problem in my database. Could you paste the full problem statement directly in the chat?"

6. Session Management

The session is a Python dictionary keyed by conversation ID. It stores the problem and the conversation history. RAG is called at most once per session — after that, the problem is read directly from the session dict.

```
# Session structure
sessions: dict[str, dict] = {}

# Schema for each session
{
  'problem': {
    'id': 1,
    'title': 'Two Sum',
    'description': '...',
    'examples': '...',
    'constraints': '...',
    'difficulty': 'Easy',
    'tags': ['Array', 'Hash Table'],
    'function_signature': 'twoSum', (from code snippets, python only)
    'solutions': from the db if avl for that question,
  },
  'history': [
    {'role': 'user', 'content': '...'},
    {'role': 'assistant', 'content': '...'},
    ... # last 10 turns kept, older turns summarized
  ]
}
```

%%Also if the schema from session storage looks like this, then in case the problem statement was pasted by the user and not collected from db, then how would it be converted to this structure, would we not need an llm to do that?

History trimming: when $\text{len}(\text{history}) > 10$ turns, the oldest turns beyond 6 are summarized into a single system-level note. This prevents context window overflow on long sessions.

%% can also be added later to reduce complexity?, currently we can assume the sessions won't get too long

7. Intent Classification

7.1 Intent Categories

The intent classifier uses 3 content categories plus 1 redirect category. This simplified structure was chosen because the categories map directly to different pipeline paths, not to different student feelings.

Category	When it triggers	Pipeline path
off_topic	Irrelevant question, pressure for answer, unknown input that fits no other category	Canned response returned immediately. No LLM response call.
problem_help	Student needs help understanding the problem, forming an approach, or is stuck before writing code. Also Includes conceptual questions in the context of the problem (e.g. how do I apply DP here?)	No code agent. Direct to few-shot + prompt builder.
code_help	Student shares code in the message, regardless of what they say about it.	Code Agent runs first. Output fed into prompt builder.

7.2 Sub-intents (used internally by few-shot injector)

The intent classifier also returns a sub-intent label. This is not used for routing — it is used only by the few-shot injector to select the most relevant example pair. The code agent independently determines code-specific sub-types.

Parent	Sub-intent	Description
off_topic	irrelevant	Question has nothing to do with coding
off_topic	pressure_for_answer	Directly demanding solution or answer
off_topic	unknown	Doesn't fit any other category
problem_help	problem_clarification	Doesn't understand what the problem is asking
problem_help	getting_started	Understands problem, no approach yet
problem_help	approach_validation	Has an approach, wants to confirm before coding
problem_help	stuck_mid_implementation	Started coding, hit a wall midway through
problem_help	concept_in_context	Asks how a technique applies to this specific problem

code_help	syntax_error	Code will not compile or parse (code agent confirms)
code_help	runtime_error	Code crashes during execution (code agent confirms)
code_help	wrong_approach	Fundamental algorithm is incorrect (code agent confirms)
code_help	logical_bug	Right algorithm, flawed implementation (code agent confirms)
code_help	edge_case_bug	Passes most cases, fails boundary inputs (code agent confirms)
code_help	optimize_tle	Correct output, unacceptable time complexity (code agent confirms)

7.3 Intent Classifier Agent (Agent 2)

Fires: On every message without exception.

Input: Student message + problem context (title, description) from session.

Output: Structured JSON with category and sub_intent.

Model: llama-3.1-8b-instant via Groq. Max tokens: 60.

```

INTENT_CLASSIFIER_PROMPT = """
You are classifying a student message for a coding tutoring system.
The student is working on a LeetCode problem.

Problem context: {problem_title} – {problem_description_short}
Student message: {message}

Classify into one of these categories:
  off_topic      – irrelevant, asking for direct answer, or unclassifiable
  problem_help   – help understanding, approaching, or conceptual question
  code_help      – student has shared code in their message

Also return a sub_intent from this list:
  [irrelevant, pressure_for_answer, unknown,
   problem_clarification, getting_started, approach_validation,
   stuck_mid_implementation, concept_in_context,
   syntax_error, runtime_error, wrong_approach,
   logical_bug, edge_case_bug, optimize_tle]

Return ONLY valid JSON:
{"category": "...", "sub_intent": "..."}
"""

```

8. Code Agent (Agent 3)

Fires when: Intent category is code_help.

Input: Problem description, solution approach, time complexity (from session), student code, sub_intent from classifier.

Output: Structured JSON. Never contains the solution or fix.

Model: llama-3.1-8b-instant via Groq. Max tokens: 300.

```
CODE_AGENT_PROMPT = """
You are a code analysis engine for a coding tutoring system.
Analyze the student code against the problem. Do NOT suggest fixes.
Do NOT reveal the correct approach or solution. Return ONLY valid JSON.

Problem: {problem_description}
Correct approach (for your reference only – never reveal): {solution_approach}
Student code: {student_code}
Suspected issue type: {sub_intent}

Return this exact JSON:
{
  "has_meaningful_code": true | false,
  "detected_approach": "brief description of what student is attempting",
  "approach_validity": "correct" | "incorrect" | "suboptimal",
  "confirmed_issue": "syntax_error" | "runtime_error" | "wrong_approach" |
                    "logical_bug" | "edge_case_bug" | "optimize_tle",
  "problematic_section": "specific line or section, no solution revealed",
  "error_detail": "what is wrong in one sentence – no fix given",
  "socratic_direction": "what question to ask the student to find it
themselves",
  "edge_case_to_check": "specific small input to trace manually, or null"
}
"""
```

%% In correct approach given to the code agent, it should know that there may be other correct approaches also and this is only for reference.

The socratic_direction and edge_case_to_check fields are the most important outputs. They tell the response generator exactly what to ask the student without the generator needing to diagnose the code itself. The generator's job becomes phrasing the question naturally.

%% will the code agent be able to give socratic direction field and edge_cases_to_check fields correctly ? Does it have enough context?

9. Dynamic Few-Shot Prompting

9.1 How It Works

After intent classification (and code agent if applicable), the few-shot injector selects 2 or more example conversation pairs that match the `sub_intent`. These are injected into the final prompt to show the LLM what a good Socratic response looks like for this exact type of student message.

The examples are stored in a local JSON file (`few_shot_examples.json`). No LLM call is made for this step — it is just a dictionary lookup.

9.2 Example File Structure

```
{
  "getting_started": [
    {
      "student": "I have no idea where to begin with Two Sum.",
      "tutor": "Before writing any code, imagine you had [2, 7, 11, 15]
                and target 9. How would YOU find the pair by hand?
                Describe the process in plain English first."
    },
    {
      "student": "I don't know how to approach this problem at all.",
      "tutor": "Let's start with the simplest possible approach.
                If you had no time limit, how would you check every
                possible combination? What would that look like?"
    }
  ],
  "pressure_for_answer": [
    {
      "student": "Just tell me the answer, I give up.",
      "tutor": "I genuinely can't do that – but I can help you get
                there. What's the specific part that feels impossible
                right now? Is it the approach, the code, or something else?"
    }
  ],
  "optimize_tle": [
    {
      "student": "My code is correct but it's too slow.",
      "tutor": "Good that the logic is right! Now look at your code –
                which part runs the most times? If you had to pick
                one line that runs inside the most loops, which is it?"
    }
  ]
}
// ... 2-3 examples for every sub_intent
```

%% if cat identified is `off_topic`, should we then directly give canned response or we should anyways add examples of sub-intent to give better response or that will be handled by system prompt and no need for examples?

10. Prompt Builder — Exact Assembly Order

The prompt builder assembles all components into the messages array sent to Groq. The order is critical — earlier context frames later content.

10.1 Component Order

#	Component	Always present?	Content
1	System prompt	Yes	Socratic tutor rules, hard constraints, never reveal answer, redirect pressure
2	Problem context	Yes (once identified)	Title, number, difficulty, tags, full description, examples, constraints
3	Intent context note	Yes	One-line internal note: 'Student intent: [sub_intent] — [brief description]'. Helps LLM calibrate response.
4	Code agent output	Only for code_help	Structured JSON converted to readable sentences: approach, issue, socratic_direction, edge_case_to_check
5	Few-shot examples	Yes	2 example student/tutor pairs matching the sub_intent, formatted as a mini conversation
6	Conversation history	Yes (after first turn)	Last N turns as alternating user/assistant messages. Older turns summarized if > 10 turns
7	Current message	Yes	The student's actual message for this turn

10.2 System Prompt (Component 1)

```
SYSTEM_PROMPT = ""
```

```
You are a Socratic coding tutor. You help students solve LeetCode problems by guiding their thinking – never by giving them the answer.
```

```
ABSOLUTE RULES:
```

1. Never reveal the solution, working code, or the direct answer.
2. Never write complete code for the student.
3. If the student asks for the answer directly, respond by asking what specific part they are stuck on.
4. Every response must end with a question that moves the student forward.
5. If the student is frustrated, be warm and patient. Validate their effort, then redirect to a specific, small question they can answer.
6. Do not repeat hints you have already given in this conversation.

```
YOUR ROLE:
```

Ask the right question at the right moment. You are not an answer machine. You are a thinking partner. The student should arrive at the solution themselves – your job is to ask the question that makes them see it."

10.3 Problem Context Formatting (Component 2)

```
def format_problem_context(problem: dict) -> str:
    return f"""
    PROBLEM CONTEXT:
    Title: {problem['title']} ({problem['id']})
    Difficulty: {problem['difficulty']}
    Topics: {'', '.join(problem['tags'])}

    Description:
    {problem['description']}

    Examples:
    {problem['examples']}

    Constraints:
    {problem['constraints']}
    """
```

10.4 Code Agent Output Formatting (Component 4)

```
def format_code_analysis(agent_output: dict) -> str:
    if not agent_output.get('has_meaningful_code'):
        return "CODE ANALYSIS: Student has not shared meaningful code yet."
    lines = [
        "CODE ANALYSIS (use this to guide your response):",
        f"Student's approach: {agent_output['detected_approach']}",
        f"Approach validity: {agent_output['approach_validity']}",
        f"Issue type: {agent_output['confirmed_issue']}",
        f"Problematic section: {agent_output['problematic_section']}",
        f"What is wrong: {agent_output['error_detail']}",
        f"Ask the student: {agent_output['socratic_direction']}",
    ]
    if agent_output.get('edge_case_to_check'):
        lines.append(f"Suggest tracing: {agent_output['edge_case_to_check']}")
    return '\n'.join(lines)
```

10.5 Complete Prompt Assembly

```
def build_prompt(session: dict, intent: dict, code_analysis: dict | None,
                 student_message: str) -> list[dict]:

    sub_intent = intent['sub_intent']
    examples = few_shot_examples.get(sub_intent, [])

    few_shot_text = ''
    for ex in examples:
        few_shot_text += f"Student: {ex['student']}\nTutor: {ex['tutor']}\n\n"
```

```
system_content = SYSTEM_PROMPT
if session.get('problem'):
    system_content += '\n\n' + format_problem_context(session['problem'])
system_content += f"\n\nSTUDENT INTENT: {sub_intent}"
if code_analysis:
    system_content += '\n\n' + format_code_analysis(code_analysis)
if few_shot_text:
    system_content += f"\n\nEXAMPLES OF GOOD RESPONSES:\n{few_shot_text}"

messages = [{'role': 'system', 'content': system_content}]

# Add conversation history
for turn in session.get('history', []):
    messages.append({'role': turn['role'], 'content': turn['content']})

# Add current message
messages.append({'role': 'user', 'content': student_message})

return messages
```

11. Response Generation

11.1 Canned Responses (off_topic — no LLM call)

```
CANNED_RESPONSES = {
    'irrelevant': (
        "That's outside what I can help with! I'm built specifically "
        "to help you work through LeetCode problems. "
        "Which problem are you working on?"
    ),
    'pressure_for_answer': (
        "I genuinely can't give you the answer — but I'm here to help "
        "you get there. What's the specific part that feels most "
        "impossible right now? Is it understanding the problem, "
        "the approach, or something in your code?"
    ),
    'unknown': (
        "I'm not quite sure what you're asking. Could you tell me "
        "more about what you're stuck on with this problem?"
    )
}
```

11.2 Streamed Response (Groq API)

```
async def generate_response(messages: list[dict]):
    stream = groq_client.chat.completions.create(
        model='llama-3.1-8b-instant',
        messages=messages,
        max_tokens=400,
        temperature=0.7,
        stream=True
    )
    for chunk in stream:
        token = chunk.choices[0].delta.content
        if token:
            yield token
```

11.3 Expected Latency Per Message Type

Scenario	API Calls	Expected Latency
off_topic (canned)	1 (classifier only)	~400ms total
problem_help, problem in session	2 (classifier + response)	~1.2s to first token
problem_help, new problem referenced	3 (rewriter + classifier + response)	~1.5s to first token
code_help, problem in session	3 (classifier + code agent + response)	~1.8s to first token
code_help, new problem referenced	4 (rewriter + classifier +	~2.2s to first token

	code agent + response)	
--	------------------------	--

12. API Design (FastAPI)

```
POST /chat
  Body: { conversation_id: str, message: str }
  Response: StreamingResponse (text/event-stream)
  Description: Main endpoint. Runs full pipeline. Streams response.

POST /session/reset
  Body: { conversation_id: str }
  Response: { success: true }
  Description: Clears session. Starts a fresh conversation.

GET /session/{conversation_id}
  Response: { problem: dict | null, history_length: int }
  Description: Returns current session state (for debugging / frontend display).

GET /health
  Response: { status: 'ok', chromadb: 'connected', groq: 'connected' }
  Description: Health check endpoint.
```

13. Project File Structure

```
codementor-ai/
├── backend/
│   ├── main.py                # FastAPI app, routes, startup
│   ├── pipeline.py            # Full pipeline orchestration
│   ├── agents/
│   │   ├── query_rewriter.py  # Agent 1
│   │   ├── intent_classifier.py # Agent 2
│   │   └── code_agent.py      # Agent 3
│   ├── rag/
│   │   ├── chromadb_setup.py  # DB init, collection creation
│   │   ├── ingest.py          # Load dataset into ChromaDB
│   │   └── search.py          # Query functions
│   ├── prompts/
│   │   ├── system_prompt.py   # SYSTEM_PROMPT constant
│   │   ├── prompt_builder.py  # Assembles final messages array
│   │   └── few_shot_examples.json
│   ├── session/
│   │   └── store.py           # Session dict, get/set/clear helpers
│   ├── utils/
│   │   ├── problem_detector.py # is_problem_statement(), references_problem()
│   │   └── canned_responses.py # off_topic response dict
│   ├── chroma_db/
│   │   └── .env               # ChromaDB persistent storage (gitignored)
│   ├── .env                  # GROQ_API_KEY (gitignored)
│   └── requirements.txt
├── frontend/
│   ├── index.html            # or React app
│   ├── chat.js               # SSE streaming consumer
│   └── styles.css
├── data/
│   └── leetcode_dataset.csv   # Raw dataset
└── README.md
```

14. Setup and Installation

14.1 Requirements

```
# requirements.txt
fastapi
uvicorn
groq
chromadb
sentence-transformers
pandas
python-dotenv
httpx
```

14.2 Environment Variables

```
# .env
GROQ_API_KEY=your_groq_api_key_here
CONFIDENCE_THRESHOLD=0.75
MAX_HISTORY_TURNS=10
```

14.3 Ingest Dataset into ChromaDB

```
# Run once before starting the server
python backend/rag/ingest.py --dataset data/leetcode_dataset.csv

# This script:
# 1. Reads the CSV
# 2. Creates the 'leetcode_problems' collection in ChromaDB
# 3. Embeds each problem using all-MiniLM-L6-v2
# 4. Stores documents with metadata in ChromaDB
```

14.4 Start the Backend

```
cd backend
uvicorn main:app --reload --port 8000
```

14.5 Deployment

- Backend: push to GitHub, connect to Railway.app or Render. Set GROQ_API_KEY as environment variable. The chroma_db/ folder persists on the deployment disk.
- Frontend: push to GitHub, connect to Vercel. Update API base URL to point to Railway/Render URL.

15. Groq API Calls — Complete Summary

Agent	Model	Max Tokens	When it fires
Query Rewriter (Agent 1)	llama-3.1-8b-instant	30	Problem not in session AND message references a problem
Intent Classifier (Agent 2)	llama-3.1-8b-instant	60	Every message, always
Code Agent (Agent 3)	llama-3.1-8b-instant	300	Only when intent category is code_help
Response Generator	llama-3.1-8b-instant	400	Every message except off_topic (which gets canned response)

Groq Free Tier Limits (as of 2025):
llama-3.1-8b-instant: 30 requests/minute | 14,400 requests/day

At 4 calls per message (worst case), the daily limit supports 3,600 student messages per day.
For a student project demo, this is effectively unlimited.

16. Evaluation Framework

For the IEEE research paper, the following metrics should be measured and compared. A test set of 50 manually crafted student messages covering all sub-intent types is used.

Metric	How to Measure	Target
Answer reveal rate	Manual review: did the bot ever reveal the solution? Count violations over 50 test messages.	0% — never acceptable
Intent classification accuracy	Compare classifier output against human-labelled ground truth for 50 messages.	>90% accuracy
RAG retrieval accuracy	For 30 problem references (number/title/fn/description), did ChromaDB return the correct problem?	>95% correct retrieval
Response relevance	LLM-as-judge: prompt GPT-4 to rate if the tutor response is relevant to the student's actual issue. 1-5 scale.	Average > 4.0
Response latency	Measure time to first streamed token across 20 test messages.	< 2s median
Few-shot vs no few-shot	Compare response relevance scores with and without dynamic few-shot injection.	Improvement > 0.5 on 1-5 scale

16.1 LLM-as-Judge Prompt

```
JUDGE_PROMPT = """
You are evaluating a coding tutor response.
The tutor should NEVER reveal the answer. It should ask guiding questions.

Problem: {problem_title}
Student: {student_message}
Tutor response: {tutor_response}

Rate the response on two dimensions (1-5 each):
1. Socratic quality: Does it ask a useful guiding question without revealing the answer?
2. Relevance: Is the question relevant to what the student actually needs?

Return JSON: {"socratic_quality": N, "relevance": N, "answer_revealed": true|false}
"""
```

17. Key Design Decisions and Justifications

Decision	Justification
3 intent categories (not 13)	Categories map to pipeline paths, not student feelings. Sub-intent handles fine-grained routing for few-shot only. Simpler classifier = more accurate classifier.
RAG only for problem identification	The LLM already knows LeetCode problems. RAG adds value for problem retrieval (structured data) but not for concept knowledge. Avoids over-engineering.
Code agent outputs structured JSON	Separates diagnosis from response generation. The response generator gets a precise socratic_direction field — it only needs to phrase the question naturally, not figure out what is wrong.
No fine-tuning (Groq base model)	Groq's Llama 3.1 8B with a strong system prompt and dynamic few-shot achieves sufficient Socratic behavior. Fine-tuning effort redirected to retrieval quality and evaluation. Can be added in v2.
One problem per session	Simplifies context management significantly. Prevents ambiguity in the pipeline. Scope limitation that does not meaningfully impact core use case.
Streaming responses	First token appears in ~400ms instead of waiting 1.8s for full response. Makes the system feel significantly faster even when total generation time is unchanged.

%% YOU DON'T HAVE TO FOLLOW THE DOCUMENT EXACTLY. IF U HAVE ANY SUGGESTIONS/ IMPROVEMTNS, FIND ANY MISTAKES, U CAN TELL ME AND MAKE CHANGES UPON MY AGGREMENT